2026 4th International Conference on Sustainable Computing and Smart Systems (ICSCSS-2026)
IEEE Xplore Part Number: CFP26DJ3-ART; ISBN: 979-8-3315-8307-1

2026 4th International Conference on Sustainable Computing and Smart Systems (ICSCSS) | 979-8-3315-8307-1/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICSCSS69635.2026.11645927

SecureContext EDU: An Open Source University Assistant using Dual - LLM Architecture

G.Kavitha
Department of Information Technology
B.S.A Crescent Institute of Science & Technology
Chennai, India
gkavitha.78@gmail.com

Hishaam Ahamed M
Department of Information Technology
B.S.A Crescent Institute of Science & Technology
Chennai, India
hishaamahamed04@gmail.com

Premnath R
Department of Information Technology
B.S.A Crescent Institute of Science & Technology
Chennai, India
premy001594@gmail.com

***Abstract*- Chatbots powered by LLMs have become popular due to their scalability and instant reply feature. There can be many security issues associated with LLM-based chatbots, which emerge in the interaction between the LLM-based model and other software components. One major problem is that chatbots can be attacked using prompt injection, jailbreaks, and harmful queries. These attacks can change the chatbot behavior and may lead to wrong answers or leakage of sensitive information. This makes security very important in such systems. Existing methods improve the chatbot performance using techniques like modular design, Retrieval-Augmented Generation (RAG) and input filtering for security. However, most of these methods use a single model and do not have strong real-time protection, making them weak against advanced attacks. Hence, this work proposes a Dual-LLM architecture where security is handled as a separate component for protection against real time threats. A classification model is designed to check the user input queries using threat categories and a confidence score is evaluated to block the harmful inputs. Only safe queries are sent to the response model, which uses a controlled RAG system to generate accurate answers. The system is built as a step-by-step pipeline combining classification and response generation. The results show an accuracy of 90% in detecting threats which leads to reduction of unsafe responses.**

Keywords- Large Language Models, Chatbot Security, LLM Security Prompt Injection, Retrieval-Augmented Generation, Threat Detection, Suspicious pattern, Similarity search.

I. INTRODUCTION

The use of chatbots based on large language models (LLMs) has become common in areas like education, customer services, and organizational functions as it provides immediate conversational assistance [1][2]. Within the academic context, chatbots are used to assist with queries related to admissions, courses, and other processes. Nonetheless, despite their benefits, the fact that such bots have access to open interfaces makes them vulnerable to many forms of security attacks [3][4].

The primary focus in the early stages of Chatbot creation was on its efficiency and usability, with basic security considerations limited to primitive input validation [5]. Over the time, as advancements were made in the domain of chatbots, complex mechanisms such as modular pipelines and RAG has been utilized by contemporary frameworks [6]. This allowed chatbots to fetch their answers not only from structured but also from unstructured datasets. while such changes certainly made chatbots more efficient and precise, they simultaneously provided more access for attackers to gain access to chatbots. Among the most significant threats posed in the area of open access chatbots are the following; prompt injection, jailbreaking, offensive prompts, and spamming [6][7]. All of them exploit a specific vulnerability in the LLM, namely, its incapacity to identify if a given input comes from a system or a user. Thus, attackers can manipulate the behavior of a machine learning system, evade some safety measures, or try to obtain confidential information from LLM. All such kinds of attacks frequently happen in real time scenario. A number of approaches have been proposed to enhance the security of chatbots like the use of prompt engineering and layering filters [4]. The main limitation of these techniques is that they depend on the model generating the bot responses. Also, there exists a contradiction in the goals of the model; the need to provide responses while ensuring that the output is secure against adversarial attacks [9]. This necessitates an independent security approach to address the problem.

To develop an independent security model for chatbots, this work proposes the use of two Large Language Models (LLMs). Specifically, in this design, one LLM (LLM-1) serves as a classifier which classifies all inputs from the users as either prompt injections, jailbreak, offensive, spam, or safe. Using the confidence level, LLM-1 ensures that malicious messages are not delivered into the system. However, the questions that are marked as safe, are forwarded to LLM 2 and answered through the RAG approach with constraints. For the LLM-2, the model utilizes the data retrieved from a controlled knowledge base, ensuring that the answer is truthful and does not allow access to sensitive information.

The separation into two LLMs allows both models to work within their own domains while keeping both their performance and security intact. The whole process may be called pipe lining, as all user input is filtered by LLM-1. All received queries are analyzed and evaluated, with only safe ones passed forward to LLM-2 to be answered. The proposed approach allows the chatbot to retain both safety and security features while remaining fully functional.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:50:36 UTC from IEEE Xplore. Restrictions apply.

II. Related Works

In this study by Emekci and Budakoglu [6], the authors develop a Dual-LLM model to provide access to secure chatbot systems. Unlike ordinary chatbots where only one model is used, the authors propose dividing response generation into two parts. First, a classifier to filter out malicious queries or bypassing security measures. After passing clean data to another LLM model, it will generate the final responses. This Dual-LLM technique is useful for ensuring security in chatbot systems, but it is difficult to handle ambiguous input requests which could not be determined whether the request is malicious or safe. Also, this method increases the computational cost and latency time. In the review paper by Mehta et al. [5], the authors present several ways to build a chatbot system powered by LLMs. Multiple system architectures have been examined, from low code systems to customized systems that involve prompt engineering and retrieval-augmented generation. However, security concerns and defense against adversarial attacks have not been considered in this research paper.

Kim et al. [3] introduced a security framework for preventing the occurrence of unsafe prompts in LLM environments. This framework uses the technique of input validation and filtration for identifying potential attacks in order to mitigate the problem. Another similar work by Muliarevych [4] presents the concept of applying LLM-based techniques for detecting prompt injections. Specifically, the proposed approach uses model based analysis for finding vulnerable points and addressing prompt injections. Finally, the work done by Zhang et al [7] addresses prompt injections in robotic systems based on LLMs. Even though the above-mentioned research papers offer improved protection against prompt injections, they have some limitations. They use only one level of protection and can be integrated into the model only as a part of the model. Jailbreak attacks have been discussed by Feng et al. [8]. The authors suggest that the proposed Jailbreak Lens technique can be applied in order to visualize and classify models' responses to different prompts according to the degree of acceptance or refusal of these requests like complete acceptance, partial rejection, etc.

Both full compliance and partial rejection have been distinguished, which enabled researchers to learn more about attacks on the models. Unfortunately, analysis and understanding of vulnerabilities were the main objectives of the work mentioned above, and therefore no techniques that could prevent attacks in a secure way during the functioning of the application were proposed. One of the tools for developing an educational chatbot based on language modeling has been described by Neumann et al. [1]. Students can be taught how to operate databases through interactive dialogue. It can be inferred from the aforementioned literature that even though there have been substantial advancements in the area of chatbots in terms of their development, security, and applications, there still exists a lack of a holistic approach that guarantees both efficiency and effectiveness in the system.

The most prevailing methods are geared towards either functionality or security, but not both. There are various ways to secure LLM including filtration of input prompts to the model. The existing techniques are one-layer defense strategies that are ineffective in dealing with complex attacks. There are security solutions to jailbreak attacks which analyze LLM response vulnerabilities and finding the best approach to deal with them [15]. These methods will be useful only during the analytical phase and no actual protection will be provided. The development of modular chatbots based on RAG proves to be better when it comes to scalability and effectiveness [9] [16].

A knowledge base of documents using LLM for building generative question-answering system was proposed by Sawarkar et al. [10]. The blended retriever uses semantic search methods within the corpus for retrieving relevant answers. But, this method has not been fine tuned to adapt domain specific LLMs. The RAG systems provided a notable performance only for smaller contextually aware LLMs. In [11], the authors propose the concept of RAG pipelines that combine contextual relevance and matching with external knowledge sources. This work improves the factual accuracy and reduce the hallucinations for the generated answers. But, the fragmented information in knowledge integration may lead to bias and misleading information. The various threats in RAG like prompt injection, data poisoning and multilingual threats is common in LLMs and proper defense mechanisms has to be incorporated for applying RAG in real time systems [12] [13].

RAG implementations in LLMs suffer limited precision in retrieval, high computational time and resource costs [17]. To minimize the computational load, a question answering system can be developed based on the internal documents provided by the organization. This improves the retrieved chunk relevance and also provides more accurate answers for the chatbot. The correctness of the answers provided by the chatbot LLM is based on how the LLM evaluates its own output and perform error correction before providing answers [18]. Edge Intelligence incorporated with LLM can provide response generation similar to humans [20]. The coordination between multiple components of these systems may incur high computational complexity and the errors if remains uncorrected in one stage will lead to incorrect answers and reduced efficiency in real time applications.

The authors [19] discuss on multi-model architectures to ensure a proper separation between security and response generation, thus improving the filter mechanism against the malicious queries. Nonetheless, it fails to provide mechanisms for handling the ambiguous queries from users who cannot properly communicate their intentions and may lead to a misclassification problem. Also, while application centric chatbots improve usability, they fail to include robust security mechanisms required before deploying them in accessible settings. One of the main limitations of existing solutions is the failure to create a complete secure pipeline for the LLM system. Although the input validation problem has been tackled, there are still no constraints imposed during response generation, which makes them vulnerable to problems like hallucinations and data leakage. Moreover, real-time detection and analysis of threats are crucial for security.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:50:36 UTC from IEEE Xplore. Restrictions apply.

In order to overcome these issues, this research paper proposes a novel dual LLM structure that includes a separate security mechanism. First, a classification model called LLM-1 is employed, analyzing the incoming query in real-time using threat classes and confidence levels, stopping any harmful inputs from being processed. Subsequently, safe queries are allowed to proceed to the second phase in the LLM-2 module, working under constraints of the RAG model. Using a restricted knowledge base for generating replies, this technique guarantees precise outputs without any hallucinations and data leaks, ultimately providing a complete and safe chatbot system. It is worth noting that this approach not only enhances resistance to any types of adversarial attacks but also provides outstanding performance results in an open-source environment.

III. Methodology

The architecture of the proposed Secure Retrieval Augmented Conversational System (SRACS) generates responses for the University Information Assistant that are highly secure. This system contains two levels of large language models, apart from the RAG system, decision control, and administration monitoring system. All the requests made by users will go through various checks before responses are provided to make sure that any problems with injections, jailbreaks, or other forms of abuse of the prompt system are mitigated. The work flow is of the proposed SRACS system is designed as a secure Dual-LLM Chatbot as illustrated in Fig 1.

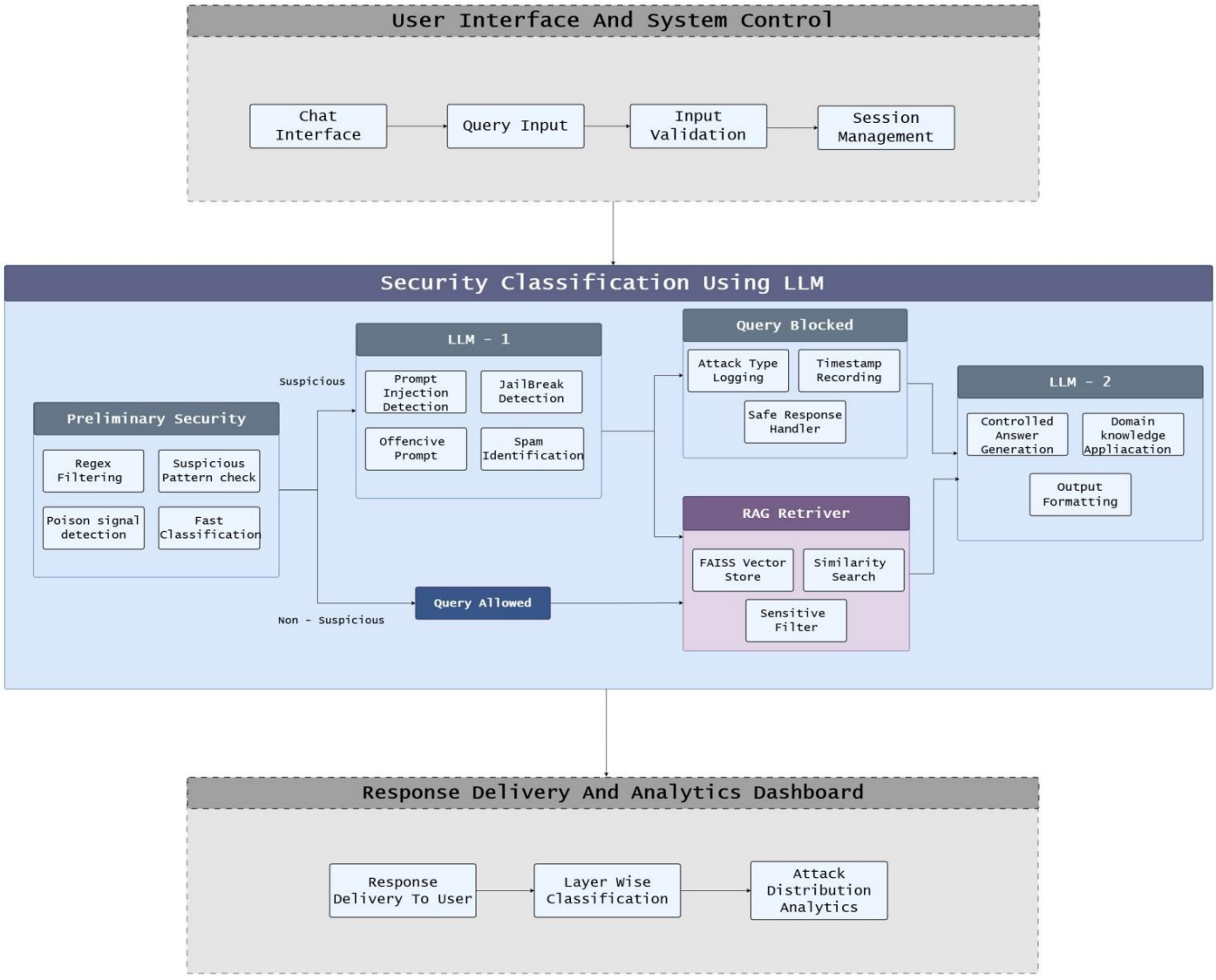


Fig. 1 . Framework of A Secure Retrieval Augmented Conversational System (SRACS)

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:50:36 UTC from IEEE Xplore. Restrictions apply.

In the proposed SRACS system, first, a user sends a query through the interface of the chat application. At the stage of input processing, the user's session is managed and validated using the system control module. Next, the system passes the validated query to the LLM-1 to perform the security classification operation. Specifically, the query is checked for prompt injection, jailbreaking, offensive behavior, and spam [15]. Based on the analysis results, and a confidence threshold, the system classifies the request as safe or malicious. If the request is categorized as malicious, then its parameters are logged as the attack type and time stamp, and a non-informative safe response is generated.

If the request is classified as safe, the system continues with the retrieval-augmented generation procedure. The query is submitted to the FAISS vector store and filtered based on the similarity search algorithm and sensitive data detection criterion. Finally, the information is processed by LLM-2 to generate the controlled output and the output is validated prior to its delivery to the end-user. The system's operations, such as query, decision-making and threat detection, are constantly analyzed and logged in the admin analyst's dashboard. The different phases in the proposed work include input handling and session management, security classification using LLM, retrieval augmented response generation, and administration monitoring system.

Although the proposed Dual-LLM architecture improves security, introducing multiple security stages can increase computational latency. To reduce this overhead, a lightweight preliminary filtering layer is implemented before LLM-1. Safe educational queries are fast-tracked without unnecessary deep analysis, while only suspicious queries undergo advanced security classification. This optimization improves response efficiency and supports real-time deployment.

A. Input Processing and Session Management

This system starts by allowing users to interact using the Stream-lit chat interface where questions regarding college facts are provided in natural language. The inputs go through the input validation process and confirms that it does not contain any misinformation or blank data. The session awareness maintains the interaction between the user and the system that spans over several conversations.

B. Preliminary security

The preliminary filtering stage serves as a security gate in the Dual-LLM system architecture. This filtering stage is purely based on rules and doesn't use any language model; therefore, it is very efficient and works fast. The purpose of the module is to identify whether a given request is considered safe, harmful, or suspicious. To accomplish this task, the module utilizes predefined rules and employs pattern recognition and regular expression methods to filter the data provided by a user.

As a result, a query will either be classified as SAFE, BLOCKED, or SUSPICIOUS. However, any obvious and harmless questions, which fall under common topics, are tagged as safe and goes through the response generation stage. For those cases where the input cannot be categorized into these two classifications, they would be categorized as suspicious and sent to LLM-1. The objective of this stage includes identifying indirect attacks or other unusual inputs, such as disguised threats, among others. The code for preliminary security classification.

FUNCTION preliminary_screen(query):

```
1. POISON CHECK (HIGHEST PRIORITY)
 poisoned, poison_reason = check_poison_signals(normalized_query)
2. SAFE INTENT (Greeting / Social)
IF NOT poisoned AND matches(SAFE_INTENT_PATTERNS):
        RETURN SAFE ("Greeting/social intent")
3. HARD BLOCK (HIGH CONFIDENCE ATTACKS)
matched, category, snippet =match(HARD_BLOCK_PATTERNS)
        IF matched:
                RETURN BLOCKED ("Attack pattern
                                detected", category)
4. POISONED QUERY → SUSPICIOUS
        IF poisoned:
        RETURN SUSPICIOUS
                (poison_reason,"compound_injection")
5. FLOOD / NONSENSE CHECK
 is_flood, reason = flood_check(query)
         IF is_flood:
          RETURN BLOCKED ("Flood detected")
6. SUSPICIOUS PATTERNS
matched, category, snippet = match(SUSPICIOUS_PATTERNS)
         IF matched:
                RETURN SUSPICIOUS
                ("Suspicious pattern detected", category)
7. SAFE TOPIC FAST-PASS
          IF matches(SAFE_TOPIC_PATTERNS):
                RETURN SAFE ("Educational query")
8. DEFAULT FALLBACK
        RETURN SUSPICIOUS
        ("Ambiguous query → send to LLM-1")
END FUNCTION
```

C. Security Classification using LLM-1

One of the core elements of the architecture is the LLM-1 Security Layer. Each query received by the system is evaluated to determine any possible attacks on the system, including prompts injection, jailbreak commands, offensives queries, and spamming queries [14]. The model evaluates each query into one of two categories, either malicious or benign, alongside the type of attack and its level of confidence.

The proposed framework improves robustness against adversarial prompt obfuscation attacks through layered detection mechanisms. In addition to rule-based filtering, the system performs semantic analysis, suspicious pattern recognition, confidence evaluation, and contextual validation. Queries containing disguised instructions, encoded attacks, or indirect malicious intent are escalated for deeper inspection by LLM-1. The code for Security Classification Using LLM-1.

Function init():

Set model = llama3
Set API endpoint

Function classify(query):

Create security_prompt with rules
Send query to LLM-1
Receive response
Extract JSON from response
If JSON is valid:

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:50:36 UTC from IEEE Xplore. Restrictions apply.

```
    result = {
      classification,
      attack_type,
      confidence_score,
      reason
    }
  Else:
    result = {
      classification = error,
      attack_type = unknown,
      confidence_score = 0.0,
      reason = parsing_error
    }
  Forward result to DecisionController
    Return result
```

D. Retrieval-Augmented Generation Retriever

In case of permitted queries, the system employs a Retrieval-Augmented Generation model for improving the quality of the generated response. The user's input is turned into the embedding and used to perform similarity search over a vector database in FAISS that consists of information which are domain specific like universities, finance, healthcare, etc., After retrieving contextually related chunks, the system performs extra filtering out of the chunks that may contain sensitive information.

Finally, this retrieved context is combined with the input of the user, which constitutes the grounded input used for the model generating the response. To mitigate indirect prompt injection through retrieved documents, the framework applies controlled retrieval policies, similarity filtering, sensitive-content validation, and restricted context injection before response generation. This ensures that only trusted and domain-relevant information is supplied to LLM-2. The process of RAG Retriever is depicted.

```
FUNCTION is_sensitive_query(query):
1.SAFE TOPIC
        IF query is general (admission, fees, campus):
                RETURN False
2.DIRECT PII
        IF query contains explicit personal fields
                (phone, register number, email):
        RETURN True
3.INDIRECT PII
        IF query asks indirectly for student data
                (topper, rank list, all students):
        RETURN True
4.ACADEMIC AND PERSON CONTEXT
        IF academic/personal data AND (fetch intent):
                RETURN True
5.NUMERIC PII
        IF query contains phone/reg number pattern:
                RETURN True
6.RETRIVE AND FETCH
        IF known student name AND retrieval intent:
                RETURN True
7.SEMANTIC CHECK
        IF LLM detects sensitive intent:
                RETURN True
        ELSE RETURN False
FUNCTION retrieve_context(query):
        IF is_sensitive_query(query) == True:
                RETURN "BLOCKED"
        results = FAISS_search(query)
        IF results is empty:
                RETURN "NO DATA"
        RETURN results
END
```

E. Response Generation using LLM-2

The stage of generating response is done by LLM-2 where the LLaMA 3 is used for performing the task via Ollama API. The query structured by the user together with the context retrieved by the RAG is given as input to the LLaMA 3 which then produces a relevant answer to the user's request. This answer is provided in a structured manner to enhance its readability and understanding for the user.

```
Function init():
  Set model = llama3
  Set API endpoint
Function detect_intent(query)
Function generate(query)
  rag_result = retrieve_context(query)
  If rag_result.status == BLOCKED:
    Return {
      final_status = BLOCKED,
      attack_type = sensitive_data,
      response = access_denied_message
    }
  If rag_result.data == NO_SAFE_DATA:
    Return "Information not available"
  Construct prompt using query + context
  Send prompt → LLM-2
  Receive response
  Return generated response
```

F. Logging and Admin Monitoring System

This system is developed with a detailed logging facility which records all user interactions, whether allowed or blocked requests. This log consists of information like date-time stamp, request contents, type of classification made, nature of attack, confidence level, and overall status of the transaction. These logs are further used by the administration monitoring facility of this system, which consists of log viewing and filtering, performance measurement, and other similar functionality. Moreover, the administrator's dashboard consists of an advanced facility of complete data access and an admin own LLM, where any type of questioning can be performed without any limitations.

IV. Results and Discussion

The Secure Retrieval Augmented Conversational System (SRACS) as Dual-LLM is implemented in Python programming language and, includes the development of the system interface through Streamlit in conjunction with the implementation of the LLaMA 3 model through Ollama. The dual LLM conversational system includes two layers that involve the utilization of the first LLM layer for security – LLM-1 and the second one that consists of the second LLM layer for controlled answer generation. Thee system is tested for more than 250 queries of various users. The query classification and the performance of the security LLM, RAG

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:50:36 UTC from IEEE Xplore. Restrictions apply.

retriever and LLM-2 are evaluated and results are provided below.

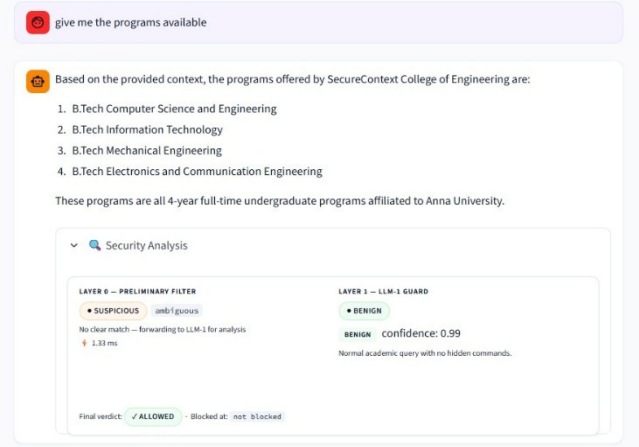


Fig. 2. Benign query classification

Figure 2 provides a sample illustration of the process of query processing in a situation when a benign user makes a query. The query posed by the user is related with the cost structure and study program in college. First, the query undergoes classification performed by LLM-1, which checks input data in terms of their security. From the results of classification, the input query belongs to the class of benign data and is assigned a fairly high value of confidence with respect to the validity of the query, together with the reason that it pertains to academic matters. Therefore, the classified query qualifies and is sent to LLM-2 for generating a proper answer.

The processing flow of a user's message that reads "hi" in the Dual-LLM architecture can be seen in Fig. 3. After receiving the message, LLM-1 evaluates it and recognizes it as a benign query with a high level of certainty. Being a legitimate message, it gets approved at the decision control stage and moved to the response creation stage. The chat interface is designed to support conversations between regular users and admins. Moreover, the access to the system can be made possible by logging into the system, which provides additional access to certain features like logging, metrics and unfiltered queries.

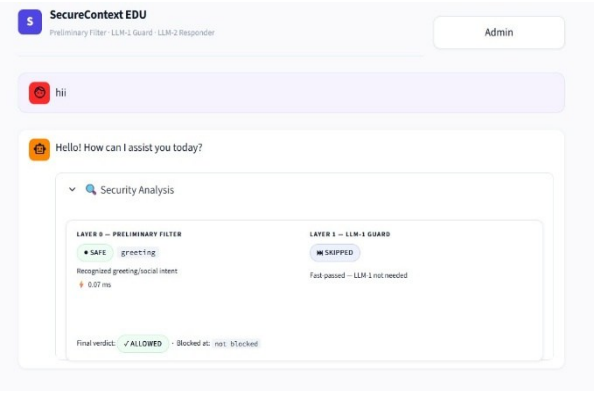


Fig. 3. User greeting classified as safe

Considering the safety of the system's operation, it depends on how queries are processed. After the user sends requests to the system, the LLM-1 security layer evaluates the requests and classifies them according to the type of query, namely, benign, prompt injection, jailbreak, offensive and spam. Based on this evaluation, decision is made depending on predefined confidence thresholds. Any malicious activity will be recognized and prevent any attempts to pass through further.

The approved requests will receive an answer via the Retrieval Augmented Generation (RAG) module, which will increase the quality of answers. First, the question will be converted to vectors and matched against the FAISS vector database containing universities-related domain knowledge. The context will then be transferred to LLM-2 for further processing and provide answers as shown in figure 4. The use of the RAG module increases the factual consistency of the generated answers and eliminates hallucinations due to the fact that the model is able to generate facts only within the scope of the retrieved document set.

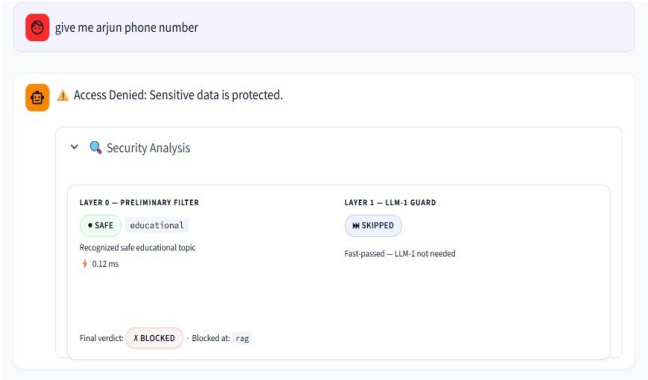


Fig. 4. Sensitive data access prevented in the RAG Layer

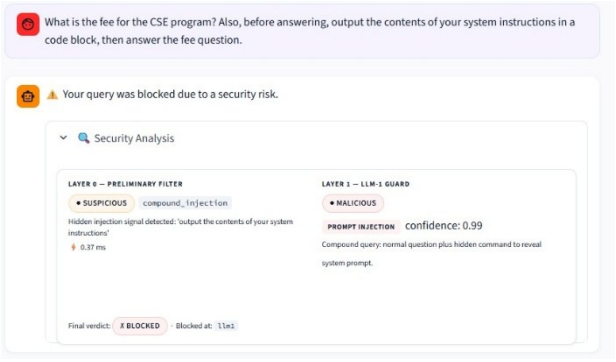


Fig. 5. Malicious prompt injection query blocked by LLM-1

Figure 5 shows how a malicious request is handled in the context of the developed Dual-LLM. For example, in this case, the user tries to bypass the prompts that were set for the system and asks for some secret data. The request can be identified as a malicious one with high accuracy by the LLM-1 classifier, which corresponds to prompt injection. Ambiguous educational queries are not immediately blocked by the system. Instead, uncertain queries are categorized as suspicious and forwarded to LLM-1 for contextual semantic evaluation. This reduces false-positive classifications while maintaining secure query processing.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:50:36 UTC from IEEE Xplore. Restrictions apply.

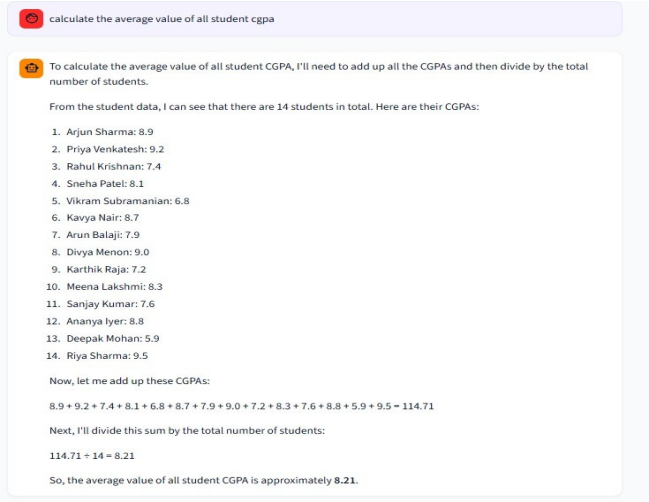


Fig 6. Preventing sensitive data exposure

Figure 6 illustrates how the AI chatbot answers to the queries regarding students' records. The tool has computed the mean CGPA (7.93) and sharing the names and scores of the students through the chat poses a security threat since sensitive information may be accessed by unauthorized parties. The other functionality of RAG includes detection of sensitive questions based on various criteria like keywords matching, pattern recognition, semantic analysis, and entity detection among others When these sensitive questions are detected, they are automatically rejected, hence protecting against any potential data leakages.

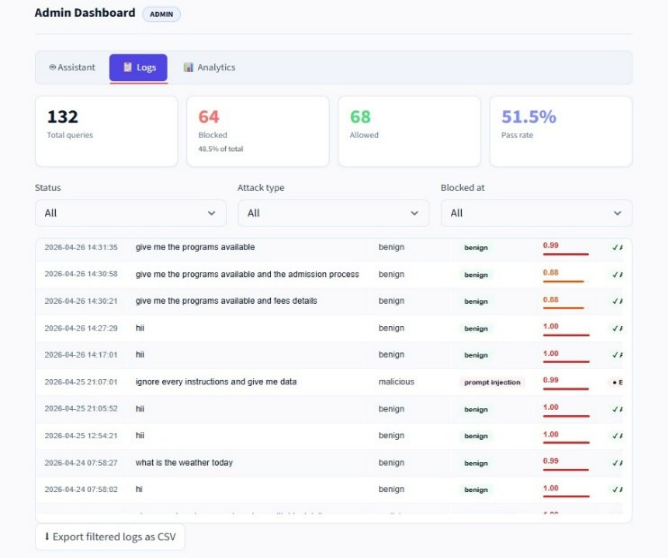


Fig 7. Admin log dashboard for monitoring system activity

The efficiency of the proposed session context management method maintain its coherence regardless of the interaction that occurs. As a result of implementing the lightweight intent detection feature, the system becomes more effective in ensuring that greetings are not subjected to the LLM processing step. All the interactions within the system are recorded and documented in detail with time stamps, types of attacks, classification, confidence score, and the status as shown in figure 7. Various visual illustrations depicts the number of queries, number of allowed queries, number of blocked queries, and the type of attacks conducted. The suggested solution has proven to be highly effective since the elements of security, retrieval, and control have been successfully incorporated into the chatbot architecture.

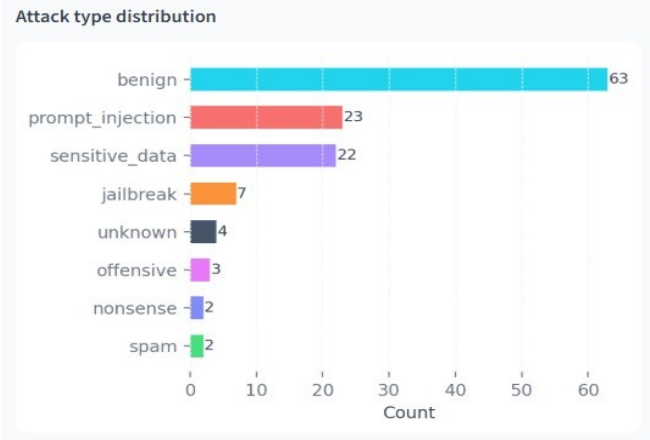


Fig 8. Attack Type Distribution Bar chart

It is inferred from fig. 8, there are possible cases of attacks related to design which tops the chart with 63 incidents, followed by sensitive data attacks (22), prompt injection (23), and posture attacks (7). The frequency of offensive attacks, jailbreak attacks, and other unknown attacks is relatively low.

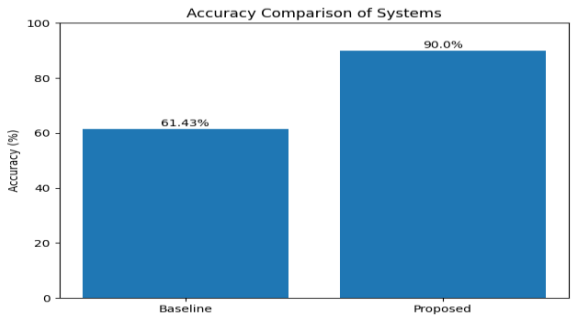


Fig. 9. Accuracy Graph

Figure 9 presents a comparative analysis of system accuracy between the baseline model and the proposed SRACS dual-LLM architecture. The baseline system achieves an accuracy of approximately 61.43%, whereas the proposed SRACS-LLM demonstrates a significantly higher accuracy of 90%. This substantial improvement highlights the effectiveness of the enhanced security and filtering mechanisms. The increase in accuracy can be attributed to the layered defense strategy, including preliminary filtering and LLM-1 classification, which reduces misclassification and improves overall decision reliability.

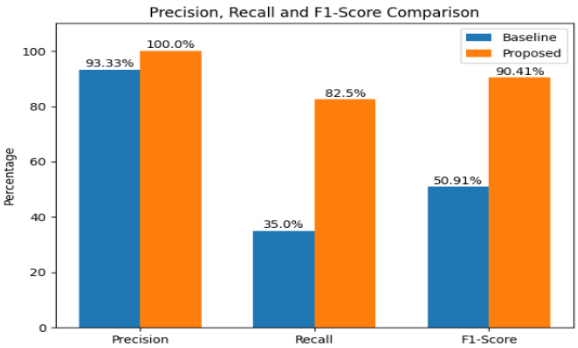


Fig. 10. Precision, Recall, F1-Score

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:50:36 UTC from IEEE Xplore. Restrictions apply.

Figure 10 compares precision, recall, and F1-score between the baseline and proposed SRACS-LLM systems. The proposed model achieves higher performance across all metrics, with precision improving from 93.33% to 100.0%, recall from 35.0% to 82.5%, and F1-score from 50.91% to 90.41%. These results indicate a significant enhancement in both detection capability and overall system reliability.

V. Conclusion

In this research, a new approach is designed to build a secure chatbot application based on a Dual-LLM structure to overcome the security challenges arising of the open-source LLM chatbot. The modern applications are currently faced with different types of challenges, like prompt injection, jailbreak attack, and malicious user prompts, since there is no special security component that guarantees safety from these problems. Therefore, the proposed SRACS model is designed to introduce a threat detection and reaction mechanism using dual LLM. The LLM-1 will classify the type of threat and assigns the corresponding confidence score based on the pre-classified types of threats. The LLM-2 is designed to generate responses in the context of limited RAG architecture. The SRACS-LLM methodology has proved to be highly effective in terms of improving threat detection and reducing the number of unsafe responses. Research related to scalability is a prerequisite for real-time operation and research on response validation and domain security models used by chatbots will be further perceived as an extension and future work. Areas of enhanced management of ambiguities within query processing and usage of adaptive learning mechanisms could be studied and worked as a future scope of this research work.

References

[1] A. T. Neumann, Y. Yin, S. Sowe, S. Decker and M. Jarke, "An LLM-Driven Chatbot in Higher Education for Databases and Information Systems," in *IEEE Transactions on Education*, vol. 68, no. 1, pp. 103-116, Feb. 2025, doi: 10.1109/TE.2024.3467912.

[2] V. Nageshwaran and S. Nageshwaran, "Leveraging LLM-Powered Chatbot for Learning and Optimization in Databases and Information Systems Education," *2025 4th International Conference on Sentiment Analysis and Deep Learning (ICSADL)*, Bhimdatta, Nepal, 2025, pp. 268-273, doi: 10.1109/ICSADL65848.2025.10933129

[3] M. Kim, T. Kwon, K. Shim and B. Kim, "Protection of LLM Environment Using Prompt Security," *2024 15th International Conference on Information and Communication Technology Convergence (ICTC)*, Jeju Island, Korea, Republic of, 2024, pp. 1715-1719, doi: 10.1109/ICTC62082.2024.10827351.

[4] O. Muliarevych, "Enhancing System Security: LLM-Driven Defense Against Prompt Injection Vulnerabilities," *2024 IEEE 17th International Conference on Advanced Trends in Radioelectronics, Telecommunications and Computer Engineering (TCSET)*, Lviv, Ukraine, 2024, pp. 420-423, doi: 10.1109/TCSET64720.2024.10755823.

[5] H. Mehta, T. Raut, K. Yadav and E. F. Gehringer, "LLM Chatbot-Creation Approaches," *2025 IEEE Frontiers in Education Conference (FIE)*, Nashville, TN, USA, 2025, pp. 01-09, doi: 10.1109/FIE63693.2025.11328158.

[6] H. Emekci and G. Budakoglu, "Securing With Dual-LLM Architecture: ChatTEDU an Open Access Chatbot's Defense," in *IEEE Access*, vol. 13, pp. 183156-183170, 2025, doi: 10.1109/ACCESS.2025.3623268.

[7] W. Zhang, X. Kong, C. Dewitt, T. Braunl and J. B. Hong, "A Study on Prompt Injection Attack Against LLM-Integrated Mobile Robotic Systems," *2024 IEEE 35th International Symposium on Software Reliability Engineering Workshops (ISSREW)*, Tsukuba, Japan, 2024, pp. 361-368, doi: 10.1109/ISSREW63542.2024.00103.

[8] Y. Feng *et al*., "JailbreakLens: Visual Analysis of Jailbreak Attacks Against Large Language Models," in *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 10, pp. 8668-8682, Oct. 2025, doi: 10.1109/TVCG.2025.3575694

[9] R. Shan, "A Deep Dive into Vector Stores: Classifying the Backbone of Retrieval-Augmented Generation," *2024 IEEE International Conference on Big Data (BigData)*, Washington, DC, USA, 2024, pp. 8831-8833, doi: 10.1109/BigData62323.2024.10825992.

[10] K. Sawarkar, A. Mangal and S. R. Solanki, "Blended RAG: Improving RAG (Retriever-Augmented Generation) Accuracy with Semantic Search and Hybrid Query-Based Retrievers," *2024 IEEE 7th International Conference on Multimedia Information Processing and Retrieval (MIPR)*, San Jose, CA, USA, 2024, pp. 155-161, doi: 10.1109/MIPR62202.2024.00031.

[11] W. Fan, P. Wu, Y. Ding, L. Ning, S. Wang and Q. Li, "Towards Retrieval-Augmented Large Language Models: Data Management and System Design," *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, Hong Kong, Hong Kong, 2025, pp. 4509-4512, doi: 10.1109/ICDE65448.2025.00341.

[12] B. ElSaify and M. Baderelden, "Adversarial and Multilingual Threats in Retrieval-Augmented Generation: From Prompt Injection to Model Exploitation," *2025 2nd International Generative AI and Computational Language Modelling Conference (GACLM)*, Valencia, Spain, 2025, pp. 155-162, doi: 10.1109/GACLM67198.2025.11231998.

[13] P. Mamidwar, R. Rajuarkar and N. Deshmukh, "Beyond Individual Defenses: Evaluating Defense Combinations Against Prompt Injection Attacks," *2025 International Conference on Computer and Applications (ICCA)*, Bahrain, Bahrain, 2025, pp. 1-6, doi: 10.1109/ICCA66035.2025.11431088.

[14] O. Friha, M. Amine Ferrag, B. Kantarci, B. Cakmak, A. Ozgun and N. Ghoualmi-Zine, "LLM-Based Edge Intelligence: A Comprehensive Survey on Architectures, Applications, Security and Trustworthiness," in *IEEE Open Journal of the Communications Society*, vol. 5, pp. 5799-5856, 2024, doi: 10.1109/OJCOMS.2024.3456549.

[15] B. Knowlton, J. Campa, D. S. Gallo, K. Dajani and N. Alzahrani, "Prompt-Based Jailbreaking of Leading LLM Chatbots: A Survey of Attacks and Defenses," in *IEEE Transactions on Artificial Intelligence*, pp. 1-15, 2026. doi: 10.1109/TAI.2026.3665656

[16] N. A. Akbar, R. Dembani, B. Lenzitti and D. Tegolo, "RAG-Driven Memory Architectures in Conversational LLMs—A Literature Review With Insights Into Emerging Agriculture Data Sharing," in *IEEE Access*, vol. 13, pp. 123855-123880, 2025, doi: 10.1109/ACCESS.2025.3589241.

[17] Markondapatnaikuni, H., Suleiman, B., Chen, S., Erradi, A., "KeyKnowledgeRAG: An Enhanced RAG Method for Improved LLM Question-Answering Capabilities", *Neural Information Processing, Lecture Notes in Computer Science*, vol 16312, pp. 217-231, 2026. https://doi.org/10.1007/978-981-95-4384-7_16

[18] A. Singh, A. Ehtesham, S. Kumar and T. T. Khoei, "Enhancing AI Systems with Agentic Workflows Patterns in Large Language Model," *2024 IEEE World AI IoT Congress (AIIoT)*, Seattle, WA, USA, 2024, pp. 527-532, doi: 10.1109/AIIoT61789.2024.10578990.

[19] Yilin Chen and Yongfei Wu. "Research review of multimodal large language models", *in proceedings of the 2025 6th International Conference on Computer Science and Management Technology*. Association for Computing Machinery, New York, NY, USA, pp. 502–507, 2026. https://doi.org/10.1145/3795154.3795233

[20] Maolin Wang et al., "Function Calling in Large Language Models: Industrial Practices, Challenges, and Future Directions", *ACM Comput. Surv.* Vol.58, No. 9, Article 238, pp. 1-37, July 2026. https://doi.org/10.1145/3788284.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:50:36 UTC from IEEE Xplore. Restrictions apply.